

Chapter 1: Databases and Database Users

Fundamentals of Database Systems — Elmasri & Navathe (7th Ed.)

Concise Exam Preparation Notes | Low Weightage Chapter

1. Core Definitions

Term	Definition
Data	Raw, unorganized facts that need to be processed (e.g., a list of numbers).
Information	Processed, organized data presented in context to make it useful.
Database (DB)	An organized collection of related data, designed for easy access, management, and updating.
Mini-world	The specific part of the real world whose data is stored in the DB (e.g., university grades).
DBMS	Software that facilitates creation, maintenance, and controlled access to a database.
Database System	The combination of DBMS software + the stored data (and sometimes the applications).
Metadata	Data about data — stored in the system catalog/data dictionary; describes structure & constraints.

□ *Exam Tip: Know the difference between Data vs. Information and DB vs. DBMS vs. Database System.*

2. Types of Databases & Applications

Traditional Applications

- Numeric and Textual Databases (earliest type)

Modern / Recent Applications

- Multimedia Databases (images, audio, video)
- Geographic Information Systems (GIS)
- Biological and Genome Databases
- Data Warehouses
- Mobile Databases
- Real-time and Active Databases
- Social Networks — Facebook, Twitter, LinkedIn (generate massive transactional data)
- Search Engines — Google, Bing, Yahoo (maintain own web-page repositories)
- Big Data systems — large clusters of distributed computers
- Cloud Databases — data in huge data centers using thousands of machines

□ *Exam Tip: Just know the categories. Detailed chapters come later in the book.*

3. File-Based Approach vs. Database Approach

File-Based System

Each application program defines and manages its own data files. Files contain records; records contain fields.

Limitations of File-Based Approach

Limitation	Explanation
Separation & Isolation	Each program has its own data; useful data in one program is unknown to others.
Duplication	Same data stored in multiple files — wastes space, risks inconsistency.
Program-Data Dependence	File structure is hard-coded in the program; changing the file breaks the program.
Incompatible File Formats	Programs written in different languages cannot easily share files.
Fixed Queries	Any new data requirement needs a brand-new program.
Poor Security	Files can be opened and modified directly; no roles or privileges.

Why the Database Approach Emerged

- Data definitions were embedded inside programs rather than stored independently.
- No central control over data access and manipulation.
- Solution: the DBMS, which separates data definition from application logic.

□ *Exam Tip: Exam-common question: List limitations of the file-based approach (any 3–4).*

4. Typical DBMS Functionality

- Define the database — data types, structures, and constraints.
- Construct / Load — store initial data on secondary storage.
- Manipulate the database:
 - Retrieval — queries and report generation.
 - Modification — insertions, deletions, updates.
 - Web access — through web application interfaces.
- Process & Share — concurrent multi-user access while keeping data valid and consistent.
- Security — user roles and access privileges.
- Active Processing — internal triggers and actions on data.
- Backup & Recovery — ensures completed transactions are permanently recorded.
- Maintenance — database software and system upkeep over its lifetime.

Applications interact with the DB via:

- **Queries:** Access and retrieve parts of data based on conditions.

- **Transactions:** Read and/or update data; must be executed correctly or fully aborted (ACID).

5. Main Characteristics of the Database Approach

1. Self-Describing Nature

The DBMS catalog stores metadata (description of data structures, types, constraints). This allows the DBMS to work with different database applications without hard-coding structure in the software.

2. Program-Data Independence

Data structures and storage can be changed without modifying application programs. This is possible because data definitions are stored in the catalog, not in the programs.

3. Data Abstraction

A data model hides storage details and presents users with a conceptual view. Programs reference the data model, not physical storage.

4. Support of Multiple Views

Different users can see different views of the same database, each showing only the data relevant to that user.

5. Sharing of Data & Multi-user Transaction Processing

- Concurrency control ensures transactions execute correctly even when multiple users access the DB simultaneously.
- Recovery subsystem guarantees that completed transactions are permanently recorded.
- OLTP (Online Transaction Processing) — supports hundreds of concurrent transactions per second.

□ *Exam Tip: These 5 characteristics are a likely short-answer question.*

6. Database Users

Category A: Actors on the Scene (interact with DB directly)

User Type	Role
Database Administrator (DBA)	Authorizes access, monitors use, manages hardware/software resources, ensures efficiency.
Database Designer	Defines content, structure, constraints, and transactions. Must communicate with end-users.
Casual End Users	Access the DB occasionally as needed, using ad-hoc queries.

Naive / Parametric Users	Large group; use pre-built 'canned transactions'. Examples: bank tellers, reservation clerks, mobile app users.
Sophisticated End Users	Business analysts, scientists, engineers; use software tools working closely with the DB.
Stand-alone Users	Maintain personal databases (e.g., personal photo DB, tax software users).
System Analysts	Understand user requirements and design canned transaction applications.
Application Programmers	Implement, test, and debug the applications designed by analysts.
Business Analysts	Analyze large volumes of data and real-time 'Big Data' for decision-making.

Category B: Workers Behind the Scene (build & maintain the DBMS)

- **System Designers & Implementors:** Design and implement DBMS modules and interfaces; integrate with OS, compilers, and applications.
- **Tool Developers:** Build software tools for DB modeling, performance monitoring, prototyping, UI creation, etc.
- **Operators & Maintenance Personnel:** Manage the hardware and software environment for running the database system.

□ *Exam Tip: Know the distinction between Actors on the Scene vs. Workers Behind the Scene, and the sub-types.*

7. Advantages of Using the Database Approach

- Controlling redundancy — avoids duplicate storage of the same data.
- Data sharing — multiple users and applications can share the same data.
- Restricting unauthorized access — roles and privileges control who sees what.
- Persistent storage for program objects — especially in OO-DBMSs.
- Efficient query processing — through indexes and storage structures.
- Query optimization — DBMS optimizes queries automatically.
- Backup and recovery services — data is protected against failures.
- Multiple interfaces — tailored views for different user categories.
- Representing complex relationships — between different data entities.
- Enforcing integrity constraints — rules that keep data correct.
- Deductive and active rules/triggers — draw inferences and automate actions.
- Enforcing standards — consistent data item names, formats, report structures.
- Reduced application development time — new apps are faster to build.
- Flexibility to evolve data structures — DB schema can change as needs change.
- Availability of current information — critical for real-time systems.
- Economies of scale — consolidating data avoids wasteful duplication of resources.

□ *Exam Tip: High-level list is enough for low-weightage chapters — know 6–8 key advantages.*

8. Historical Development of Database Technology

Era	Development
Mid-1960s–1970s	Hierarchical and Network Models introduced. IBM's IMS (hierarchical) still widely used.
1970	Relational Model proposed (by E.F. Codd at IBM Research).
Early 1980s	Relational DBMS products launched commercially.
Late 1980s–Early 1990s	Object-Oriented DBMSs (OODBMSs) introduced for complex data (CAD etc.) — limited adoption.
1990s–2000s	Object-Relational DBMSs (ORDBMSs) emerged, combining relational + OO features.
2000s	Web data in HTML; E-commerce using XML; PHP/JavaScript for dynamic web pages tied to DBs.
2010s–Present	Big Data (petabytes/day), Hadoop, MapReduce, Google File System, Apache Spark, NOSQL systems.

9. Extending Database Capabilities

New Application Areas

- Scientific Applications (Physics, Chemistry, Biology, Genetics)
- Earth, Atmospheric Sciences, Astronomy
- XML — eXtensible Markup Language for structured/semi-structured data
- Image, Audio, and Video Data Management
- Data Warehousing & Data Mining (major growth area)
- Spatial Data Management & Location-Based Services
- Time Series & Historical Data Management

Big Data & NOSQL

- **Big Data:** Deals with data at petabyte scale (10^{15} bytes). Technologies: Hadoop, MapReduce (from Google/Yahoo), Spark.
- **NOSQL (Not Only SQL):** Designed for rapid search/retrieval from documents, large social-network graphs, and unstructured data with flexible transaction models.

10. When NOT to Use a DBMS

Cost Factors (Inhibitors)

- High initial investment in hardware and software.
- Overhead from providing generality, security, concurrency control, recovery, and integrity.

Situations Where DBMS May Be Unnecessary

- Database and applications are simple, well-defined, and unlikely to change.
- No requirement for multi-user access.

Situations Where DBMS May Be Infeasible or Insufficient

- Embedded systems — general-purpose DBMS may not fit in available storage.
- Stringent real-time requirements — DBMS overhead may cause unacceptable delays (e.g., telephone switching systems).
- Data complexity exceeds DBMS modeling capabilities (e.g., complex genome/protein databases).
- Special operations needed that the DBMS does not support (e.g., GIS-specific spatial operations).

□ *Exam Tip: 'When not to use a DBMS' is a classic exam question — memorize the 3 categories.*

Quick Reference Summary

Topic	Key Points to Remember
Data vs. Information	Data = raw facts; Information = processed/organized data.
DBMS Core Role	Define, Create, Maintain, and Control access to the database.
File-Based Limitations	Redundancy, isolation, program-data dependence, incompatible formats, fixed queries, poor security.
DB Approach Characteristics	Self-describing, program-data independence, data abstraction, multiple views, data sharing/OLTP.
User Categories	DBA, DB Designer, Casual, Naive/Parametric, Sophisticated, Stand-alone, System Analyst, App Programmer, Business Analyst.
Key Advantages	Reduced redundancy, shared access, security, integrity, backup/recovery, standards, fast development.
History Milestones	1965: Hierarchical/Network; 1970: Relational (Codd); 1980s: RDBMS products; 1990s: OODBMS/ORDBMS; 2000s: Web/XML; 2010s: Big Data/NOSQL.
When NOT to use DBMS	Simple/static apps, no multi-user need, embedded systems, real-time constraints, unsupported complex data.